

↳ Recursion
 ↳ technique where we call funⁿ within itself
 base case is true.
 function recursive() {
 base condition:
 return();
 ↳ Repetitive tasks
 ↳ stored in call stack

Recursion
 ↳ define a funⁿ
 ↳ define a base condition
 ↳ if count is false, recursion will not happen
 ↳ else if count is true, it will end recursive funⁿ by doing.

Basic Multiple
 function add(3) {
 add();
 }
 Time complexity
 $O(n)$
 ↳ exponential values
 function sample(3) {
 sample() + sample + sample.
 }
 Time complexity $O(3^n)$
 ↳ e.g. fibonacci

Recursion vs Loops

for or in range(1, 20) {
 }
 ↳ faster
 ↳ less memory usage
 Disadvantage
 ↳ difficult to read.
 ↳ more code.

function add(3) {
 add(2);
 add(1);
 add(0);
 }
 Time complexity
 ↳ concise code
 ↳ easy to understand
 Disadvantage
 ↳ slow
 ↳ stack overflow

Basis

Call stack
 calc-fact(5) → return 1
 calc-fact(4) → return 1
 calc-fact(3) → return 1
 calc-fact(2) → return 1
 calc-fact(1) → return 1

Multiple
 0, 1, 2, 3, 5, 8, 13, 21, 34, ...
 + + + + +
 n = 6

fibonacci
 fib(6) = fib(5) + fib(4)
 fib(5) = fib(4) + fib(3)
 fib(4) = fib(3) + fib(2)
 fib(3) = fib(2) + fib(1)
 fib(2) = fib(1) + fib(0)
 fib(1) = fib(0) + fib(-1)

n = 6
 (n-1) = 5
 25
 81
 0, 1, 1, 2, 3, 5, 8

sum of digits
 n = 1432
 1 + 4 + 3 + 2 = 10
 [2 + 3 + 4 + 1]

divisor
 1432 / 10 = 143 R 2
 143 / 10 = 14 R 3
 14 / 10 = 1 R 4
 1 / 10 = 0 R 1
 10 / 10 = 1 R 0
 10 / 10 = 1 R 0

2 + 3 + 4 + sum of digits (10)
 2 + 3 + 4 + sum of digits (10)
 2 + sum of digits (143)
 sum of digits (1432)

$(n \% 10) + \text{sum of digits}(n / 10)$

Math cases

↳ Tree Traversal
 ↳ sorting algo [quick, merge]
 ↳ search, finds in a subarray.
 ↳ generally or check possible permutation and combination
 ↳ parenthesis (balanced or not)

[2, 4, 6, 8, 10, 12] , target = 9